What is Prompt Engineering?

- Method to design, optimize prompt to use with large language model with the most efficient
- The good prompt engineering that means good conversation
- Reference: https://www.kaggle.com/whitepaper-prompt-engineering

Prompt Engineering is the method of designing and optimizing prompts to use with Large Language Models (LLMs) in the most efficient way. Good prompt engineering means good conversations with AI that produce accurate, consistent, and useful results.

The 5 Key Components of a Good Prompt

1. Role / Persona

- Defines the AI's perspective and expertise level
- Example: "You are an expert in formal argumentation..."
- Effect: Uses correct technical language, follows domain conventions

2. Task

- Clearly states what the AI must do
- Example: "Extract all topics from this hotel review and classify sentiment"
- Tip: Use action verbs - Extract, Classify, Generate, Transform

3. Context

- Provides background information the AI needs to understand the task
- Example: "This review is from Booking.com. Topics include: Room, Staff, Location..."
- Why it matters: Context prevents misinterpretation and guides the AI's focus

4. Examples / Shots

- Demonstrations that guide the model's output style and format
- Example: Review: "Great breakfast, terrible wifi" → Output: {"food": "positive", "facility": "negative"}
- Power: Examples teach the AI the exact pattern you want, reducing ambiguity

5. Output Format

- Specifies exactly how the result should be structured
- Example: "Return ONLY valid JSON with no markdown, no backticks..."
- Critical for automation: Ensures AI output can be parsed programmatically

BONUS: Constraints

- Limiters or requirements that make output more precise (like a limiter/requirement of output)
- Examples:
  - No more than 300 characters
  - Do not use the word 'very'
  - Use only vocabulary from the original text

Q&A

- Is the role really effective to the output?
  - Role/Persona will only affect output in case we are setting a style for a LLM output
  - Role acts as a context-setting layer for AI to choose tone, perspective, or mindset
  - For correctness of answers, it's almost the same

Technique 1: Zero-Shot Prompting

What it is

- Prompt without any example — just 2 from 5 components (Task + Output)
- Use most often
- Used to benchmark and check the quality of LLM models

When to use

- Testing model capabilities
- Benchmarking different models
- Simple, straightforward tasks

Example

Extract all topics from this hotel review: "The room was clean but the staff was rude."
→ Expected output: Room (positive), Staff (negative)

Pros & Cons

- Fast and simple
- Good for testing
- Works well with strong models
- Less consistent results
- May misinterpret complex requests
- Format may vary

Technique 2: Few-Shot Prompting

What it is

- Give model a few examples when prompting to guide the model
- Provide 2-5 examples to guide the model toward the desired output pattern
- Helps model understand the desired output format and style

When to use

- Need consistent formatting
- Complex pattern recognition
- Specific interpretation needed

Example

Example 1: Review: "Great breakfast, terrible wifi" Output: {"food": "positive", "facility": "negative"} Example 2: Review: "Helpful staff, noisy location" Output: {"staff": "positive", "location": "negative"} Now analyze: "The room was spacious but outdated"

Best Practices

- Use 2-5 examples (sweet spot)
- Make examples diverse
- Keep examples simple and clear
- Show edge cases in examples

Technique 3: Chain-of-Thought (CoT) Prompting

What it is

- Make the model thinking like a step — fits logic work or calculation

- Stimulates the model to show intermediate reasoning steps before giving the final result
- Easiest use: add "Let's think step by step" to your prompt
- Most generative AI now have this behind the scenes — like GPT-4 showing 'Thinking'

Magic Phrase

"Let's think step by step" — this simple phrase activates reasoning mode!

When to use

- Complex logic problems
- Multi-step tasks
- Mathematical calculations
- When you need transparency in reasoning

Example

Determine if this review supports "good_staff" claim. Let's think step by step: Review: "The receptionist was helpful but forgot room keys twice." Step 1: Identify staff-related statements → Step 2: Determine positive vs negative evidence → Step 3: Weigh the evidence → Step 4: Make final decision

Why it's powerful

- Accuracy increases by 20-40% for complex tasks!
- Reduces logical errors
- Makes reasoning traceable
- Helps catch mistakes

Technique 4: Few-Shot Chain-of-Thought

What it is

- Give an example that has reasoning steps, then ask a new task and tell the model to follow that example
- Combines Few-Shot + CoT by providing examples that include reasoning steps

Example

Extract ABA rules from reviews: Example: Review: "Staff went above and beyond" → Reasoning: 1. Topic is "staff", sentiment is positive 2. Head: "good_staff" 3. Support: "went_above_and_beyond" 4. Contrary: "no_evident_not_went_above_and_beyond" → Output: good_staff ← went_above_and_beyond, no_evident_not_went_above_and_beyond. Now analyze: "Room was spotless and well-maintained"

Best for

- Teaching complex patterns
- Domain-specific reasoning
- Your ABA mining tasks!

Technique 5: Self-Consistency

What it is

- Generate multiple reasoning paths and select the most consistent answer
- Generate multiple responses for the same prompt, then select the most common answer (majority vote)
- Improves accuracy by considering different perspectives
- Works well with Chain-of-Thought for complex reasoning tasks

When to use

- Critical decisions
- Reducing hallucination
- When accuracy is paramount

How to implement

- Set temperature > 0 (e.g. 0.7) for variety
- Run same prompt 3-5 times
- Collect all responses → Count most frequent answer → Use the majority response

Example

Prompt: Extract sentiment for 'location' Run 1: Positive | Run 2: Positive | Run 3: Negative | Run 4: Positive | Run 5: Positive → Result: Positive (4/5 votes)

Technique 6: Tree of Thoughts (ToT)

What it is

- Explores multiple reasoning branches like a decision tree
- Evaluates different paths and backtracks when needed
- Useful for complex problem-solving requiring exploration of alternatives

When to use

- Complex problem-solving
- Multiple valid approaches exist
- Need to explore alternatives

Example

Task: Extract rules from ambiguous review. Review: "Location convenient for business but noisy at night" → Generate 3 paths: Path 1: Focus on positive (good_location) → score: 3/10 | Path 2: Focus on negative (bad_location) → score: 3/10 | Path 3: Split into two separate rules → score: 10/10 → Select: Path 3

Perfect for handling ambiguity in hotel reviews!

Technique 7: Meta-Prompting

What it is

- Use AI to generate prompts — the AI becomes your prompt engineer
- Reference: https://jmiemirza.github.io/Meta-Prompting/

When to use

- Creating prompt templates for routine work
- Optimizing existing prompts
- Don't know how to start

How to use

- Give a very wide question to LLM and tell them to help write a new prompt — e.g. "Please help me create a prompt for building a Python function to count websites starting with letter G in the homepage of a Search Engine"

Concept

- Main concept has 2 components: Task (what we want) + Output (what the result looks like)
- Generative AI will create the rest of the components itself — Context, Persona, Examples
- Very useful when you need to create a template prompt for routine work

How it works

- Give AI a broad question
- Tell it what components you need
- AI generates Task, Context, Examples, etc.
- You refine the generated prompt

Technique 8: Expert Prompting

What it is

- Technique to prompt LLM that aims to use AI with full potential by setting it as an expert
- Add 1 more component to the prompt: an expert identity — so the LLM knows an expert of that kind first
- Reference: https://arxiv.org/html/2305.14688v2

When to use

- Domain-specific knowledge needed
- Technical terminology is important
- Following conventions matters

Example

> *You are an expert in formal argumentation and assumption-based argumentation (ABA) frameworks. You have published papers on argument mining and knowledge representation. You understand the precise requirements of ABA formalism. Given this hotel review, extract the ABA structure following these rules: [Your detailed extraction rules]*

Why it works

- The role primes the AI to choose appropriate tone
- Uses technical vocabulary specific to the domain
- Follows domain conventions
- Applies relevant knowledge more accurately

Technique 9: Structured Output Prompting

What it is

- Request specific output formats like JSON, XML, or tables
- Force the AI to return output in a specific, parseable format — critical for automation
- Makes it easier to parse and use AI responses programmatically

Critical for

- Your ABA mining tasks!
- Any automated pipeline
- Programmatic processing

Example

> *Return ONLY valid JSON. No markdown, no preamble, no backticks: {"head": "good_staff", "supports": ["helpful", "friendly", "professional"], "contraries": ["no_evident_not_helpful", "no_evident_not_friendly", "no_evident_not_professional"]} Review: "The staff were helpful and friendly"*

Key Rules

- Say 'ONLY valid JSON'
- Say 'No markdown'
- Say 'No backticks'
- Say 'No preamble'
- Provide the exact format you expect

Testing tip: Always test if output can be parsed with JSON.parse()!

Technique 10: Iterative Refinement

What it is

- Start with a basic prompt and gradually refine based on outputs
- Use feedback loop to improve prompt quality
- Break complex tasks into sequential steps, refining output at each stage

Example

> *Step 1: Extract topics from review → Output: ["staff", "room"] Step 2: For each topic, extract sentiment → Output: {"staff": "positive", "room": "negative"} Step 3: Generate head for each → Output: {"staff": "good_staff", "room": "bad_room"} Step 4: Extract supports from review text → Output: {"good_staff": ["helpful", "friendly"]} Step 5: For each support, generate contrary → Output: Complete ABA structure*

Why powerful

- Each step is simple and verifiable
- Easier to debug
- Can validate intermediate results
- Higher overall accuracy

Technique 11: Constrained Generation

What it is

- Add specific constraints to control output
- Apply strict constraints to force AI to follow specific rules — vocabulary, format, length, etc.
- Helps ensure outputs meet specific requirements or standards

Example

Extract supports from review using ONLY these constraints: 1. Use vocabulary from original text (NO synonyms) 2. Remove articles (a, an, the) 3. Convert to present tense 4. Lowercase only 5. Replace spaces with underscores 6. Keep phrases short (2-4 words max) 7. No adjectives/adverbs unless critical Review: "The staff were exceptionally helpful and went out of their way" Correct: helpful_staff, went_out_of_way Wrong: friendly_staff (synonym), very_helpful (adverb)

Benefits

- Prevents hallucination
- Maintains source fidelity
- Ensures format compliance
- Critical for formal systems!

Technique 12: Multi-Stage Validation

What it is

- After each generation step, validate the output before proceeding to the next step
- Catches errors early before they propagate

Example

Stage 1: Extract topics → Output: ["staff", "room", "location"] Stage 2: Validate — Do these topics appear in review? → staff, room, location (not mentioned) → Corrected: ["staff", "room"] Stage 3: Extract rules for validated topics only Stage 4: Check — Does each support use review vocabulary? → Remove any supports with synonyms

Why crucial

- Catches errors early
- Prevents error propagation through pipeline
- Higher final accuracy
- Transparency in process

Task 1: Multi-label Topic Modeling

- Goal: Extract topics from hotel reviews and classify sentiment for each
- Input: Hotel review text
- Output: List of (topic, text, sentiment) triples
- Topics: Room, Staff, Location, Food, Price, Facility, Check-in, Check-out
- Use structured output for consistent formatting
- Apply few-shot prompting with domain-specific examples
- Recommended technique stack: Few-Shot + Structured Output + Constraints

Task 2: Inference Rule Extraction

- Goal: From topic-relevant text, extract ABA inference rules (head + supports + contraries)
- Input: (Topic, text, sentiment) from Task 1
- Output format: head ← support1, support2, contrary1, contrary2
- Example: good_staff ← helpful, friendly, no_evident_not_helpful, no_evident_not_friendly

- Subtasks:
  - 2a. Generate head (automatic) — Positive: good_{topic} | Negative: bad_{topic}
  - 2b. Extract supports (needs AI) — Short phrases from review using original vocabulary only
  - 2c. Generate contraries (pattern-based) — Positive: prefix 'no_evident_not_' | Negative: prefix 'have_evident_'

- Use Chain-of-Thought to show reasoning process
- Expert prompting with formal logic expertise
- Recommended technique stack: Expert Prompting + Few-Shot CoT + Constrained Generation

Task 3: Contrary Mining

- Goal: Determine all valid contraries for each assumption in the ABA framework
- Input: Set of assumptions from Task 2
- Output: For each assumption, a set of contraries that attack it
- Use contrastive prompting to highlight differences
- Apply vocabulary-constrained generation for specific terms

- Challenge — most complex task because:
  - Multiple valid contraries may exist
  - Requires understanding logical contradiction
  - Context-dependent results

- Recommended technique stack: Self-Consistency + Tree of Thoughts

Interest prompt Technique 1: Cascading Validation Prompting

Concept

- Use a two-agent approach: Generator creates output, Validator checks it
- If validation fails → regenerate with error feedback
- Multiple validation steps in sequence, each step verifies different aspects
- Reduces hallucination and improves accuracy

Why it's novel

- Most research uses single-pass generation
- Validation catches format errors automatically
- Mimics human review process

Implementation

Generator Prompt: "Extract ABA rule from review..." ↓ Output: rule ↓ Validator Prompt: "Check if this rule follows ABA requirements: 1. Head format: (good|bad)_{topic} 2. Supports use original vocabulary 3. Each support has one contrary 4. No synonyms Rule: {generated_rule} | Review: {original_review} Output: {valid: true/false, errors: [list]}" ↓ If valid: accept | If invalid: regenerate with error feedback

Expected Results

- 10-15% improvement in format compliance
- 5-10% reduction in hallucination

Interest prompt Technique 2: Vocabulary-Constrained Generation

Concept

- Pre-extract vocabulary from review, then force AI to use ONLY those words
- Any other word is flagged as [UNKNOWN]
- Particularly useful for ABA mining with formal terminology

Why it's novel

- Prevents synonym substitution
- Maintains source fidelity
- Critical for legal/medical/formal domains
- Directly addresses the paper's main constraint

Implementation

Step 1: Extract vocabulary Review: "The staff were exceptionally helpful" Extract: ["staff", "exceptionally", "helpful"] Step 2: Generate with constraint Prompt: "Generate supports using ONLY: staff, exceptionally, helpful. If you need other words, mark as [UNKNOWN]" Output: helpful_staff, [UNKNOWN_exceptional]_staff Step 3: Human review of [UNKNOWN] flags → Decide: Keep original or remove

Expected Results

- 20-30% reduction in vocabulary violations
- Near 100% source fidelity

This technique solves a major problem in the paper: GPT-4o sometimes uses synonyms instead of original words.

Interest prompt Technique 3: Contrastive Prompting for Disambiguation

Concept

- When sentiment is ambiguous, generate multiple interpretations explicitly
- Document reasoning for each, then select the best one
- Present contrasting examples to highlight differences
- Helps model understand what to include vs. exclude
- Effective for identifying contradictions and conflicts

Why it's novel

- Handles mixed sentiment explicitly
- Documents reasoning process
- Aligns with inter-annotator disagreements in research

Example

Review: "Location convenient for business but noisy at night" Interpretation A (Positive focus): Head: good_location | Supports: convenient_for_business Interpretation B (Negative focus): Head: bad_location | Supports: noisy_at_night Interpretation C (Split into two rules): Rule 1: good_location ← convenient_for_business | Rule 2: bad_location ← noisy_at_night → Captures both aspects separately → Selection: Interpretation C (most complete)

Expected Results

- 15-25% better handling of mixed-sentiment reviews
- More complete ABA frameworks

Interest prompt Technique 4: Hierarchical Task Decomposition with Confidence

Concept

- Break task into subtasks, assign confidence score to each
- Flag low-confidence results for human review
- Solve each subtask independently then combine results
- Improves handling of complex multi-step problems

Implementation

Subtask 1: Extract topics → Output: ["staff", "room"] | Confidence: 0.95 Subtask 2: Classify sentiment → Output: {staff: positive, room: negative} | Confidence: 0.87 Subtask 3: Generate heads → Output: {staff: good_staff, room: bad_room} | Confidence: 1.0 Subtask 4: Extract supports → Output: {good_staff: ["helpful", "friendly"]} | Confidence: 0.73 FLAG FOR REVIEW Threshold: 0.80 → Action: Human reviews Subtask 4

Benefits

- Provides transparency in which steps are uncertain
- Identifies weak points automatically
- Enables active learning
- Efficient use of human review time

Evaluation Metrics

1. Micro-Precision & Micro-Recall (from the paper)
- Micro-Precision = Correct elements / Generated elements
- Micro-Recall = Correct elements / Gold standard elements
- Then average across all instances for global view

2. Vocabulary Fidelity (NEW — propose this)
- Vocabulary Fidelity = (Supports using only review words) / (Total supports)
- Target: > 95%

3. Format Compliance (NEW — propose this)
- Format Compliance = (Outputs matching ABA structure) / (Total outputs)
- Checks: Head format (good|bad)_{topic}, each support has exactly one contrary, correct contrary prefix, no duplicate supports
- Target: > 90%

4. Ambiguity Handling Success Rate (NEW — propose this)
- For reviews with mixed sentiment: Success = System correctly identifies and handles both sentiments
- Compare: Single-pass vs Contrastive Prompting

5. Human Agreement — Cohen's Kappa
- Classic metric: Compare AI output to expert annotations
- Cohen's Kappa > 0.8 = Strong agreement